

贷后信贷风控与处置

信衡 CreditSentry

从风险预警到授信处置
的多 Agent 可举证自主闭环

银行不缺预警系统。 缺的是预警之后那一段。

谁去取证、凭什么下结论、敢不敢自动执行、出了事怎么向监管交代——
这四个问题没答案，再准的预警也只能停在「给人看的建议」。

场景与价值

目标场景来源、目标用户、核心痛点、真实需求、业务价值、输入输出与行业可复制性。

一位风险经理的一天

预警系统很勤快，人跟不上。

1,200+ 每日新增预警 重复推送与例行提醒占大头	4 核实一条要跨的系统 征信 · 司法 · 工商 · 流水	3 天 单案平均处置周期 取证、写报告、走审批	0 可复核的完整证据链 依据散在聊天记录与经验里
--	---	---	--

真风险淹在噪声里，误报又把正常客户抽贷断贷。

场景怎么定位到的，给谁用

我们没有从「哪个环节适合上 AI」出发，而是从一条断裂的作业链上找断点。

① 预警产生 银行已有系统	② 核实取证 人工跨系统查	③ 风险定性 人工判断	④ 出处置方案 人工撰写	⑤ 审批 必须留给人	⑥ 执行与留痕 人工 + 事后补材料
------------------	------------------	----------------	-----------------	---------------	-----------------------

目标用户

- 贷后风险经理 主用户。每天面对成百上千条预警，逐条核实
- 客户经理 协同方。只有他能向客户索取财务报表这类不在任何系统里的材料
- 合规与内审 监督方。他们不看结论，看的是「这个结论怎么来的」

真实需求 · 不是「更准的预警」

- 把噪声压下去，但不能漏高危——被丢弃的每一条都要能回溯。
- 把证据固定下来——事后能翻回原件，而不是「我记得当时看过」。
- 敢自动执行的那部分要分级——哪些能自主、哪些必须人批、哪些永远不碰。
- 每个结论都要能指回具体证据与具体规则——内审问得出，我们答得上。

四个痛点，两种性质

前两条是效率问题，后两条是责任问题——责任问题解决不了，效率做得再好也上不了生产。

效率 · 01

预警过载

日均千级，逐条人工核实，客户经理被淹没。

效率 · 02

取证割裂

四套系统、四套口径、四套授权，证据分散且难固定。

责任 · 03

不敢自动执行

压降额度直接影响客户，错一次就是投诉与问责，于是宁可全人工。

责任 · 04

无法举证

内审问「当时凭什么这么判」，答不上来。

一条预警进去，一套可举证的处置材料出来

系统的对外契约。左边是它需要什么，右边是它每次都必须交付什么。

输入

- **预警信号** 来源 · 主体 · 类型 · 时间 · 详情（可多条、可重复）
- **主体标识** 统一社会信用代码 / 行内主体号
- **取数授权** 征信查询授权编号。缺失即判为证据缺口，不降级绕过
- **决策时点** 可选。回溯场景下冻结时点，只喂入该日之前的信息

输出 · 9 个文件

- **处置意见书** 正文每处结论带证据角标，可点开看原件
- **审计报告** 合规项逐条举证 + 风险模式沉淀
- **证据账本** 只可追加，含哈希、来源、等级与定级理由
- **全链路轨迹** 含路由决策与裁决依据两类专有记录
- **取数留痕** 含被拒调用——越权尝试也必须留痕
- **运行指标** 降噪率 · 证据充分度 · 质疑覆盖率 · 用量与成本

结局 A

风险成立 出分级处置方案，需审批的停下等人，批准后执行并回执

结局 B

风险不成立 出「维持原状 + 加强监测」并留痕——看过并决定不动也要有据可查

结局 C

未能定论 出取证任务清单转人工，不给风险结论——证据不足时给结论就是编

业务价值分两层

第一层容易量化但不稀缺；第二层难量化，却决定这套系统能不能真的上生产。

18 ms 单案端到端（规则推理） 对照：人工以 天 计	62.5% 降噪率上限（三条实测链路） 每条丢弃都可回溯	¥ 0.015 单案模型成本 日均 2000 件 ≈ ¥ 31/天	30+ 单案自动取数次数 跨 4 个系统，全部留痕
--	--	---	---

每个结论都能指回证据 无源结论会让报告拒绝生成，而不是 打印出来	每次分支都能说清为什么 路由决策带规则编号与版本，不靠日 志拼凑	越权尝试也留痕 只在成功路径留痕，答不了「有没有 人试过绕过去」	不可逆动作永不自动执行 这一条让「自动化」在信贷场景变得 可谈
--	--	--	---

成本口径取自真实的上下文装配结果而非经验估值；不含修复轮——该发生率要接真实模型跑过才有实数。

换个行业，什么能带走，什么必须重写

四个场景共享同一组骨架。可迁移的是骨架，不可迁移的是判定口径。



保险核赔

报案归并去重 → 单证取证 → 定损定性 ⇔ 反欺诈质疑 → 分级赔付

小额自动、大额人核，与 L0-L3 天然对应

供应链金融

贸易背景真实性核验：多源单据交叉验证 + 对抗质疑

处置动作换成放款闸门

工业设备运维

告警降噪 → 多源取证 → 根因定位 ⇔ 反驳 → 分级处置

远程复位可自主、停机必须人批

内容与账号风控

信号聚合 → 证据固定 → 判定 ⇔ 申诉视角质疑 → 分级处置

限流可逆、封禁不可逆

换行业要改的是取数接口与判定口径；**架构、闸门、账本、审计不动。**

方案设计

系统架构、Agent 分工、任务拆解、协作流程、上下文传递、结果验证与异常分支处理。

那为什么不直接上大模型？

因为信贷处置动作要动客户额度。不可复现，就不可审计。

让模型自由编排

同一个案子重跑十次，可能走出七条不同流程。

内审问「为什么这次多查了一步」——没人答得上来。

结论好坏先不论，过程本身就不可复核。

我们的做法

流程骨架用确定性状态机，12 条路由规则显式声明。

模型只在阶段内做判断，不决定流程走向。

每一次分支都带规则编号与版本，可直接读出来。

代价是它不够聪明——不会自己想出新流程。

但在信贷处置这个场景，这个交换很划算。

我们的答案

1 个 Manager + 1 个 Team Leader + 6 个职能 Worker 的端到端闭环



五层系统架构

金色 = 人工必经点，红色 = 有副作用或不可逆的位置。全图只有这几处，是刻意收敛的结果。



贯穿三层的脊柱是一份权限矩阵。同一份声明同时驱动四处——运行时取数授权、Skill 授权、Worker 配置与身份文件的生成、以及回归断言。测试逐字节比对磁盘配置与矩阵生成结果，因此「权限矩阵即部署配置」是可校验的事实，不是修辞。

为什么是 6 个 Agent，不是 3 个或 10 个

把《商业银行内部控制指引》的「不相容职务分离」编译成权限矩阵，数量就数得出来。

5

+

1

=

6

权限等价类

目标函数对立拆分

职能 Worker

01

内部只读聚合

不触外部、不下结论

02

唯一 PII 取证触点

触点唯一，脱敏范围才可收敛

03 · 唯一拆分处

零工具纯推理

权限完全相同，因目标对立而拆

04

唯一写触点

全系统只有它能改额度

05

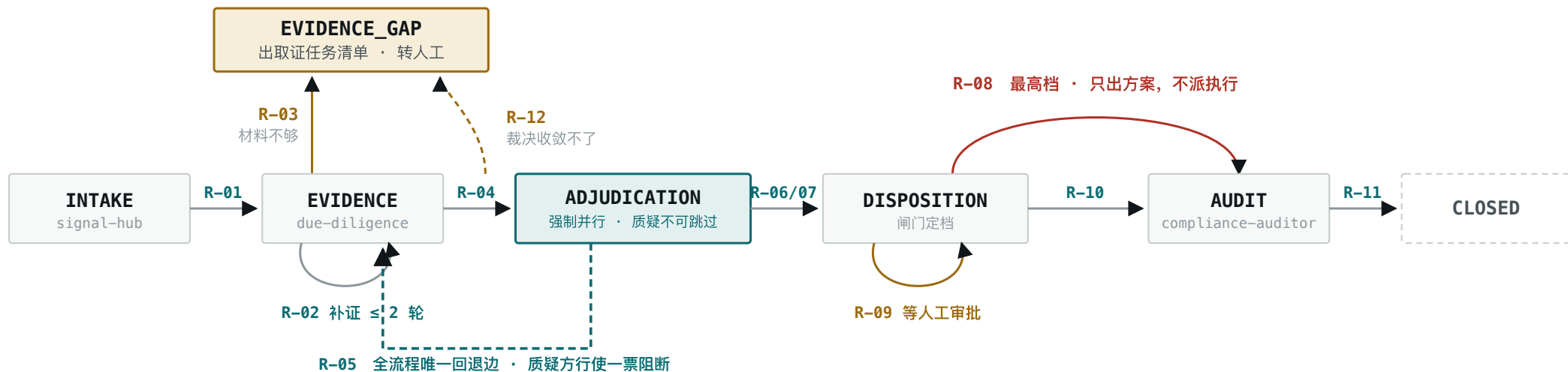
唯一审计视角

看得见全链路，动不了业务系统

合并任意两个会怎样： 定性者拿到取证工具 → 边查边下结论的确认偏差 · 定性与质疑合并 → 自己质疑自己，对抗坍塌 · 执行者兼审计 → 直接违反不相容职务分离

一个案子怎么拆成任务，谁在什么时候接手

五阶段状态机 + 12 条路由规则。模型只做阶段内判定，不决定流程走向。



任务怎么拆

按**阶段**拆而不是按步骤拆。每阶段由路由表决定派给谁、能否并行——取证阶段一个 Worker 串行取四类材料，裁决阶段两个 Worker **强制并行**。

谁来接手

Team Leader 只做路由与裁决，**不持有任何业务工具**。派活依据是路由键（阶段 · 信号类型 · 证据充分度 · 风险等级 · 敞口），不是模型的自由发挥。

可审计性的落点

路由决策**本身就是一条轨迹记录**，带路由键、命中规则编号、规则版本。「为什么当时走了这条分支」是直接读出来的，不是靠日志拼凑。

上下文传递：三个坑，逐个用代码堵

最容易出问题的不是「装不下」，而是装丢了却看不出来。三条堵法都留痕，不留痕等于没堵。

坑 一 · 静默截断

超预算时悄悄丢尾部，模型看不到关键事实却照样给结论。

堵法 裁剪必须留痕：丢了什么、为什么丢，都记进装配账单并进轨迹。必需块不参与裁剪，本身超预算则直接抛错，**不硬塞**。

坑 二 · 原文灌入

裁判文书全文进上下文，既爆预算，又让「模型看的是哪一份」无法举证。

堵法 证据只放编号 + 抽取字段，检测到长文本直接抛错。**原文靠哈希锚定，给人看不给模型看。**

坑 三 · 关键块被挤掉

预算压力下把清单、输出契约裁掉，模型不知道自己要做完什么。

堵法 排序原则是**目标与契约在前、事实在中、参考在后**。把「做完的标准」挤到中段是最糟的情况。

待办清单：不是提示技巧，是完成性契约

清单的根本弱点是**它通常由模型自己维护**——模型可以偷偷改、悄悄划掉。

所以我们的清单是：**由系统从上游产物派生 → 注入提示词告知标准 → 模型返回后由代码逐项核对**。提示词里的清单只是告知契约，真正起作用的是事后那次核对。

这堵住了一个真实漏洞：质疑方原本可以只反驳 3 条主因里的 1 条、对另外 2 条保持沉默。**未被质疑的主因 ≠ 质疑通过**。

六维查询改写

立场（求证方找构成要件，证伪方找认定门槛）· **术语规范化**（行内口语 → 监管术语）· **信号主题** · **条款直查**（精确引用不走语义）· **否定式**（仅证伪方，主动找除外与豁免）· **时效**。

时效维修复了一个真实缺陷：2017 年的回溯案件曾召回 2025 年才生效的行内制度。这是知识维度的前视污染，比证据层更隐蔽——条款看起来「一直都在」。

让 Agent 敢说「这不是风险」

比让它报警难得多。误杀正常客户会直接抽贷断贷、引发投诉与监管问责。

风险定性官

目标：论证**风险成立**。

零工具权限——刻意剥夺，防「边查边下结论」。

每条结论必须挂载证据编号，无源结论直接被拒。

对抗质疑官

目标：论证**风险不成立**。

输入完全相同、上下文完全独立、目标刻意对立。

对自动执行有一票**阻断权**。

质疑不可跳过

写在路由表里的硬约束，不交给模型判断。

两侧必须异构

绑定不同技术路线的模型。同源则「唱反调」变成自说自话，**配错直接拒绝启动**。

覆盖率参与裁决

未被质疑的主因不算通过——前者只是没人看过。

零人工，不等于无限自动化

风险等级 × 证据等级 × 敞口金额 × 可逆性 → 四档。纯规则，不联网、不问模型。

L0 只读诊断 全自主。打标、出建议，不碰业务系统。	L1 轻度干预 全自主。发通知、要材料，可撤回。	L2 影响授信 必须人工审批后执行。压降额度、追加担保。	L3 不可逆或大额 只出方案，系统永不执行。提前收贷、诉讼保全。
---	---	---	---

同一动作，不同档位 「压降 30%」在敞口 620 万时是 L2，敞口 2 亿时升为 L3。	失败方向 任何入参缺失或规则未命中，一律降为最高管控档，而不是「没匹配上就放行」。	审批被驳回 立即降级 L0 只读诊断，客户额度不被修改。
--	---	--

865 组全组合遍历证明：不可逆动作在任何入参组合下都不会落进自主档。

一个系统的可信度，看它出问题时往哪个方向倒

三条失败路径，降级方向**恒为阻断**，不为放行。

失败 一

模型给不出合规输出

定性失败 → 判「证据不足」回退补证
质疑失败 → 判「质疑未完成」**直接阻断**

质疑器坏了不等于质疑通过。

失败 二

取证不足以定论

补证 ≤ 2 轮，仍不足则转人工——**且必须有交付物**。

出《取证任务清单》：为什么停、已查到什么、还缺什么、**每项找谁补**。

失败 三

人工审批被驳回

立即降级只读诊断，客户额度不被修改。

可在取数留痕里核对：**没有任何一次成功的写操作**。

我们把所有失败路径都调成了「停下来、说清楚、交给别人」，
而不是「尽力而为地跑完」。

Skill 与工具集成

核心 Skill 设计、复用价值，工具 / 云产品 / 企业系统集成路径，以及 MCP、RAG、可观测能力规划。

16 个 Skill 分三层，分层依据只有一句话

「这件事换个行业还成立吗」——不成立的必须自研，成立的没必要重复造。

L3 领域判断 · 12 个，全自研

SignalFusion · ExposureMapping · CreditReportProbe · LitigationProbe · TxnFlowAnalyze · GuaranteeProbe · QueryRewrite · RiskRootCause · RiskGate · ComplianceCheck · SafeDisposition · PostmortemDistill

承载信贷领域知识与监管条款绑定。**不可能复用现成能力**——「单笔涉诉标的占敞口 ≥5% 且未结案才构成偿债能力信号」只能自己写。

L2 领域封装 · 4 个，自研外壳

EvidenceLedger · PolicyRag · CaseMemory · ReportCompose

语义自研，底层存取复用官方能力。账本的分级规则与哈希校验是自研的；对象存储与索引直接用云能力。

L1 云操作 · 直接用官方

对象存储 · 向量检索 · 日志 · 监控

不重复造轮子。**全部标注开源等价替代**（MinIO / pgvector / Loki / Prometheus），PoC 默认走替代，本地即可复现。

CreditSkill Spec · 让 Skill 可分发、可评估、可回滚

每个 Skill 交付三件套：**说明文档 + 输入输出 Schema + 评估集**，三者都由代码里的元数据生成——因此**文档不会与实现漂移**。

每个 Skill 显式声明：触发条件、依赖、**失败策略**、安全等级、绑定的监管条款、评估集规模、复用价值。

评估集是发布门禁

不全绿不允许经 Nacos 灰度发布。

覆盖度差额如实记录：906 组用例全绿，但其中 865 组来自闸门的全组合遍历；还有 9 个 Skill 的评估集仍是种子集、未接执行器——这些在清单里记为 SKIPPED，**不把「没跑」粉饰成「通过」**。

集成路径：Mock 与真实接入零 Schema 差异

Demo 用 Mock 是常态，风险在于「Demo 能跑、接真系统要重写」。我们用结构约束消除这个风险。

MCP SERVER	工具	性质
信贷核心	额度 · 敞口穿透 · 押品 · 担保台账 / 调额 · 追加担保 · 回滚	含写操作
人行征信	征信报告 · 期间变动比对 · 被查询记录	高敏 PII
司法工商	涉诉检索 · 裁判文书原文 · 工商登记 · 变更历史	只读
账务流水	流水明细（自动分片）· 对手方汇总 · 异常模式识别	PII
共 17 个工具。征信类必须带授权编号，缺失即拒绝，不降级绕过。		

替换点只有一处

把进程内直连换成经网关的 MCP 客户端。工具名称、入参 Schema、返回结构、错误码 完全不变——Agent、Skill、路由表与契约测试均无需改动。

为什么能做到 所有调用统一收敛在一处，因此三件事只实现一次： 权限校验、审计留痕、失败策略。

失败策略也统一 只有限流类错误值得重试； 权限与授权类错误立即上抛，不重试不降级。

企业系统落地 Higress 统一出口与凭证透传，Worker 只持消费方令牌， 真实密钥留在网关侧；PII 出站脱敏因取证触点唯一而范围可收敛、可验证。

RAG 四项能力全做，可观测多两类专有记录

赛题要求 RAG 至少两项。我们做了四项，因为少任何一项都会在这个场景出问题。

RAG · 两库四能力

- **查询改写** 六维子查询，各带独立过滤与权重；每条命中记录由哪几维召回
- **时点过滤** 条款生效日 ≤ 案件时点，防知识维前视污染
- **结果融合** 多维召回后排序融合；空集时区分「库里没有」与「被时效过滤掉了」
- **经验回流** 确认的风险模式写回案例库，可追溯哪次案件产生了哪条经验

沉淀的不只是「什么算风险」，还有「什么不算」——风险模式里专门有一条反例字段。

可观测 · 10 类轨迹

对齐 OpenTelemetry GenAI 语义约定，属性命名保持一致——接真实 Collector 时只需替换记录器，不改调用方。

另加两类本方案特有的记录：

routing 路由决策：路由键 · 命中规则编号 · 规则版本

adjudication 对抗裁决：双方结论 · 分歧点 · 采信理由

这两类是「为什么当时这么判」能被读出来的原因。通用 APM 不会记这些，因为它们是**业务语义**而非技术指标。

治理规划 · Nacos

Skill、提示词、路由表**版本化**，支持标签灰度与一键回滚。提示词版本号进轨迹——复核三个月前的处置结论时，能重建当时的全部输入。

网关规划 · Higress

统一出口、凭证透传、PII 出站脱敏。配置里凭证一律是**引用**而非明文，因此配置可进版本库、密钥不进。

这一层的进度

已实现 4 个 Mock Server（含限流、授权缺失等错误码）· 统一授权与留痕 · 10 类轨迹落盘 · 凭证引用四种形态

未实现 真实行内系统接入；Nacos 与 Higress 尚在配置层预留，网关侧未落地

把「信贷」两个字去掉，还成立的部分

这是我们挑选开源内容的唯一标准。成立的才有被别人拿走的价值。

组件	说明	协议
Agent 拆分法则	数量由权限等价类推导的方法论	CC BY 4.0
EvidenceLedger	证据账本，六个里最通用的一个	Apache-2.0
RiskGate	四维执行闸门内核	Apache-2.0
CreditSkill Spec	Skill 规范三件套 + 评估门禁	Apache-2.0
4 个 MCP 契约	金融取数接口，含契约测试	Apache-2.0
案例回放数据集	含误报陷阱样例与反事实配对	CC BY 4.0

可平移到哪些场景

保险核赔 报案归并 → 单证取证 → 定损 ⇔ 反欺诈质疑 → 分级赔付

供应链金融 贸易背景真实性核验，处置动作换成放款闸门

工业设备运维 告警降噪 → 根因定位 ⇔ 反驳 → 远程复位可自主、停机必须人批

内容与账号风控 限流可逆、封禁不可逆，天然适配四维闸门

可迁移的是骨架：信号归并 → 证据固定 → 对抗式判定 → 按可逆性分级处置 → 留痕审计。

不可迁移的是判定口径——它绑定具体监管条款，换行业必须重写。

可行性与落地

评估指标、可运行 Demo 计划、安全边界、风险控制、开放与开源计划及边界。

三条链路，三种结局

A 与 B 是同一类主体、完全相同的三类信号，差别只在个案细节。

链路 A · 风险成立 涉诉 3 笔 · 集中转出 480 万 · 法代变更 三类信号相互印证，质疑逐条尝试反驳但均不成立。 L2 需审批 额度压降 30% + 追加担保 审批通过后执行，回执与审计留痕	链路 B · 风险不成立 同样三类信号 但逐笔下钻：涉诉已结案且我方为原告、转出对手方是稳定供应商、法代变更早于风险窗口。 L0 维持原状 出「加强监测」结论并留痕—— 「看过并决定不动」也必须有据可查	链路 C · 真实历史回溯 2017 年担保圈风险传染 时点冻结在 2017-04-01，只喂入该日之前的公开信息。 L3 只出方案 直接敞口 6.5 亿触发大额升档 系统全程不执行任何写操作
---	---	--

链路 B 才是技术难点。让 Agent 敢说「这不是风险」，比让它报警难得多。

链路 C · 真实历史案例回测

把时钟拨回 2017 年 4 月，它看出来了吗



系统当时的判定

风险成立 · 关注类

净未覆盖代偿敞口达直接敞口的 2.02 倍

但档位是 L3

直接敞口 6.5 亿触发大额升档，只出方案不执行。历史上该主体当时确有阶段性缓释（覆盖 54.6%）——方向对不等于处置对。

怎么保证没作弊

每条证据标注首次公开日并逐条校验；历史结局在数据构造时即被摘出，任何环节结构上取不到。

它现在就能打开，而且可以被当场检验

python3 poc/serve.py --open

零依赖、零云账号、零网络。

最有说服力的三个动作

- **点开任一证据 → 翻开原件**
裁判文书、征信报告直接展开，并当场重算防篡改校验码
- **审批时它真的停下来等你**
后端线程阻塞在那里，不是走过场的确认框。驳回 → 降级只读，额度未被修改
- **改一个案情要素 → 当场翻案**
结论随之改变。改了却不变，说明它不是从证据推出来的

界面上看得到什么

左栏 预警池 · 判断方式 · 案情要素调节

中栏 五阶段流水线随处置推进逐段点亮；证据、对抗视图、闸门定档、执行回执按顺序长出来

右栏 办理过程 · 办理日志 · 取数留痕 · 岗位与权限

另外四个动作：并排看定性⇌质疑的逐条分歧 · 敞口改 2 亿看档位升级 · 让零权限岗位去改额度看它被拒**且留痕** · 接自己的大模型 API

工作台与命令行是**同一套代码、同一份产出**，没有为演示另造一条链路。

评估指标：怎么证明它真的有用，而不是看起来有用

准确率在这个场景没有意义——「一律报警」也能有高召回。主指标必须防得住偷懒策略。

主指标 · 配对翻转率

同一主体、同样的信号类型，只改一个决定性因子，结论必须随之翻转。

配对数据集能测**决策是否由正确的因子驱动**。系统在一对样本上给出相同结论，说明它压根没看那个因子——不管准确率多高，都是失败。

它防得住偷懒 「一律报警」与「一律放行」的翻转率**都是 0**。目标 $\geq 95\%$ ，已在工作台做成可现场操作的动作。

对照基线（复赛执行）

B1 单 Agent 直判 · B2 去掉质疑官 · B3 纯规则引擎

预期结论提前写在这里：B1 在召回上大概率与本方案接近，胜势应出现在 **误杀率、配对翻转率、证据完备率、可举证性**四项。若 B1 在这四项也不差，说明立论有问题，**应如实报告而不是换指标**。

过程指标 · 每次运行自动产出

降噪率	归并后保留条数 / 入池条数 实测 50—62.5%，且每条丢弃可回溯
证据充分度	低于 0.7 不得进入裁决阶段 实测 0.875—0.933
质疑覆盖率	目标恒为 100%，不满即回退补证 未被质疑的主因不算通过
无源结论数	目标恒为 0 出现即报告拒绝生成
修复轮次数	模型输出不合规的发生率 质量信号，不该被吞掉
单案成本与时延	按 Agent 粒度分解 用于推算规模化年成本

安全边界与风险控制

每一条都对应一段可执行的断言。带反向验证的不只测「合法输入能通过」，还测「非法输入真的会被拒」。

权限与职责边界

- 写权限唯一：只有一个角色能改额度
- PII 触点唯一：只有一个角色能读征信与流水
- 定性岗零工具权限（防边查边下结论）
- 执行方与审计方不是同一角色
- 编排者不持有任何业务工具
- 越权调用一律拒绝，且被拒调用进留痕
- 部署配置与权限矩阵零漂移

执行与数据边界

- 不可逆动作在**任何**入参组合下都不落进自主档
- 入参缺失时降为最高管控档，而非放行
- 白名单外动作一律拒绝
- 幂等重放不重复执行；回滚可恢复原值
- 审批被拒即降级只读，额度不被修改
- 无证据 / 引用无效证据的结论一律被拒
- 原文不入模型上下文；配置无明文密钥

模型使用边界

- 对抗双方必须异构，同源即**拒绝启动** 反向验证
- PII 触点的模型不得越出数据驻留白名单 反向验证
- 无模型入口的角色不得绑定模型档位
- 两种推理模式产出**同一套输出契约**
- 归一化绝不伪造证据引用 反向验证
- 模型失效时降级方向恒为阻断
- 密钥只存本次运行内存

回溯伦理边界

只回溯**已有公开定论**的历史事件，**不对当前存续且未定论的主体输出风险结论**。新闻报道只用于定位案件与时间线，不进证据账本——这恰好符合我们自己的证据等级规则。

数据来源披露

合成数据 + 公开数据，不含任何真实银行客户数据。 回溯链路逐字段标注来源——**用真实公开事件做骨架，只在法律禁止公开的两个模态上做受约束的合成。**

不是承诺，是可以现场跑的断言

每一条约束都由回归测试强制

74	906	865	13	36	0
安全边界回归 全绿	Skill 评估用例 全绿	闸门全组合遍历 穷举非抽样	接大模型的 故障模式回归	可逐条复核的 验收断言	第三方运行时 依赖

写权限唯一 只有一个角色能改额度	PII 触点唯一 只有一个角色能读征信与流水	越权尝试也留痕 被拒的调用同样进审计日志	配置零漂移 部署配置与权限矩阵逐字节比对
---------------------	---------------------------	-------------------------	-------------------------

其中带「含反向验证」的断言不只测「合法输入能通过」，还测「非法输入真的会被拒」——只测正向的约束等于没有约束。

逐条映射到赛题要求的能力

不是「用到了」，而是「用在哪、怎么验」。

能力	我们怎么用	怎么验证它真的成立
AgentTeams	1 Manager + 1 Team Leader + 6 Worker；身份、权限、模型绑定全部声明式	配置由权限矩阵生成，磁盘文件与矩阵逐字节比对，漂移即测试失败
Skill	16 个。领域判断 12 个全自研，通用云操作复用官方并标注开源等价替代	三件套（文档 + Schema + 评估集）全部由代码元数据生成；评估集是发布门禁
MCP	4 个 Server、17 个工具，覆盖信贷核心 / 征信 / 司法工商 / 账务流水	Mock 与真实接入零 Schema 差异；中央授权 + 审计留痕，越权调用被拒并留痕
RAG	监管政策库 + 历史案例库；六维查询改写、时点过滤、结果融合、经验回流	条款生效日 ≤ 案件时点，防知识维前视污染；每条命中记录由哪几维召回
可观测	10 类轨迹，对齐 OTel GenAI 语义；另加路由决策与对抗裁决两类专有记录	「为什么走了这条分支」直接读出来——带路由键、规则编号与版本
治理与网关	Nacos 版本化与灰度回滚；Higress 凭证透传与 PII 出站脱敏	凭证一律是引用而非明文，配置可进版本库、密钥不进配置层已预留，网关侧未落地

还没做的，写在明处

把差额写清楚，比事后被追问更经得起看。

已完成

6 Worker 的身份与配置声明 · 五阶段状态机与 12 条路由 · 16 个 Skill · 4 个 MCP Server · 证据账本与原文快照 · 三条端到端链路（含真实历史回溯）· 接大模型的输出契约层与失败策略 · 转人工交接单 · 风险处置工作台

尚未做

真实行内系统接入（仍为 Mock，接口按零差异设计）
生产级并发与多租户隔离
9 个 Skill 的评估集仍为种子集
Nacos 与 Higress 网关侧未落地
接大模型已在故障注入端点跑通全部 13 个模式，**但尚未在真实商业端点上跑过**

一处已知的标定局限

敞口升档阈值目前对所有客户分层用**同一套绝对值**。
在大型集团客户上，会把几乎所有可逆动作推到最高档。
机制是对的，标定不对。
这是跑真实案例才暴露的问题，如实记在这里而不是悄悄调参掩盖。

复赛计划

接入真实 Nacos 做灰度回滚演练 · Higress 落地凭证透传与 PII 脱敏 · 评估集扩至 100+ 标注案例 · 故障注入演练扩展到系统侧 · 敞口阈值按客户分层参数化 · 扩展至「贷后 + 反欺诈 + 法务」三 Team 协同

主指标：配对翻转率

同一主体、只改一个决定性因子，结论必须随之翻转。这个指标**无法靠「一律报警」或「一律放行」刷分**——两种偷懒策略的翻转率都是 0。

这套系统的价值不在于它跑得多快， 而在于它出问题时往哪个方向倒。

银行敢不敢让 Agent 动客户的额度，取决于三件事：结论有没有据、越权能不能挡、出错会不会自己停下来。
这三件事我们都做成了可以现场检验的断言。

打开工作台

```
python3 poc/serve.py --open
```

跑三条链路

```
python3 poc/run_demo.py --all
```

跑安全回归

```
python3 poc/test_safety.py
```